

Project Report on

Face Recognition Based-Attendance System

for Artificial Intelligence-(UCS411)-2425EVENSEM

Submitted By:

Aditya Pandey	(102303968)
Yash Vardhan Singh	(102303725)
Mehul Kujur	(102303712)

Submitted To:

Dr. Vijay Kumari



Department Of Computer Science and Engineering
Thapar Institute Of Engineering and Technology
(May 2025)

1. Introduction

1.1 Background

The Face Recognition Attendance System is an AI-based application designed to automate the attendance tracking process in educational institutions. Traditional attendance systems rely on manual entry, which is time-consuming and prone to errors. This project leverages facial recognition technology to identify students and mark their attendance automatically, ensuring accuracy and efficiency.

1.2 Motivation

Manual attendance systems are inefficient and can lead to errors such as proxy attendance. A face recognition-based system eliminates these issues by providing a secure, automated, and contactless solution. Additionally, it reduces administrative workload and ensures real-time attendance tracking.

1.3 Objective

The primary goal of this project is to develop an **automated, contactless, and efficient attendance system** using **face recognition technology**. The system aims to replace traditional manual attendance methods with a **secure, accurate, and scalable** solution.

Key Objectives

(i) Automated Attendance Tracking

- Eliminate manual entry by **automatically detecting and recognizing students** in real-time.
- Ensure **proxy attendance prevention** by relying on biometric verification.

(ii) High-Accuracy Face Recognition

- Implement **Haar Cascade** for robust face detection.
- Train an **LBPH (Local Binary Patterns Histogram)** model to recognize registered students with **>85% accuracy**.
- Optimize performance under varying lighting conditions.

(iii) User-Friendly Interface

- Develop an **intuitive GUI** using **Tkinter** for administrators to:
 - Register new students.
 - Manage attendance records.
 - Export/import data in **CSV format**.

(iv) Database Integration

- Store student details (roll number, department, email, etc.) in a **MySQL database**.
- Maintain a **secure and searchable** attendance log.

(v) Scalability & Future-Readiness

- Ensure the system can **handle large datasets** (hundreds of students).
- Design a modular architecture for **future upgrades**, such as:
 - **Deep learning-based recognition** (e.g., CNN).
 - **Mobile/cloud integration**.

(vi) Real-World Applicability

- Test the system in **classroom environments** to validate reliability.
 - Minimize hardware dependencies (works on standard **webcams & laptops**).
-

2. Related Work (with Comparisons)

Several existing systems use biometric methods such as fingerprint scanning or RFID for attendance tracking. However, these methods have limitations:

- **Fingerprint Scanning:** Requires physical contact and can fail due to poor sensor quality.
- **RFID Cards:** Can be lost or shared, leading to proxy attendance.

Compared to these, **face recognition** offers:

- **Contactless operation** (hygienic and convenient).
- **High accuracy** with modern AI models.
- **Difficult to spoof** (unlike cards or fingerprints).

Other face recognition attendance systems exist, but many lack:

- A **user-friendly interface** for easy management.
- **Database integration** for storing student details.
- **Real-time processing** with a webcam.

This project addresses these gaps by providing a complete solution with a **GUI**, **MySQL database**, and **real-time face detection**.

3. Proposed Technique(s) and Algorithm(s)

The system uses the following techniques:

3.1 Face Detection

- **Haar Cascade Classifier:** Detects faces in real-time using OpenCV's pre-trained model (`haarcascade_frontalface_default.xml`).

3.2 Face Recognition

- **LBPH (Local Binary Pattern Histogram):** A lightweight yet effective algorithm for face recognition, implemented using `cv2.face.LBPHFaceRecognizer_create()`.

3.3 Attendance Management

- **CSV & Database Integration:** Attendance logs are stored in attSheet.csv and can be exported/imported.
- **MySQL Database:** Stores student details (roll number, department, year, etc.).

3.4 Workflow

1. **Student Registration:** Admin adds student details via GUI.
2. **Face Data Collection:** Captures multiple face samples for training.
3. **Model Training:** Uses LBPH to train on collected images.
4. **Real-Time Detection:** Recognizes faces and marks attendance automatically.
5. **Attendance Export:** Generates CSV reports.

4. Data Set(s) Used with Description

4.1 Student Data

- Stored in a **MySQL database** (facerecognition_att).
- Fields: Roll_No, Department, Email, Year, Semester, Subject, Photo_Sample_Status.

4.2 Face Dataset

- Collected images stored in the data/ folder.
- Filename format: user.{Roll_No}.{Image_ID}.jpg.
- Each student has **100 grayscale face samples** (for training).

4.3 Attendance Logs

- Stored in attSheet.csv.
 - Fields: Roll_No, Department, Year, Time, Date, Status.
-

5. Experiments and Results

5.1 Evaluation and Results

- **Accuracy:** Achieved **~85-90% recognition accuracy** under good lighting conditions.
- **Speed:** Processes **10-15 FPS** on a standard webcam.
- **Database Integration:** Successfully links recognized faces with student records.

Comparison with Manual Attendance

Feature	Manual System	Proposed System
Accuracy	Low (human error)	High (AI-based)
Speed	Slow (manual entry)	Real-time
Proxy Risk	High (friends can mark)	Low (biometric)
Scalability	Limited (paper-based)	High (digital)

5.2 Error Analysis

- **False Positives:** Occurs due to **similar-looking faces** or **poor lighting**.
- **False Negatives:** Happens when the system fails to recognize a registered student (e.g., due to **glasses/beard changes**).
- **Dependency on Lighting:** Performance drops in **low-light environments**.

6. Conclusion and Future Work

6.1 Conclusion

The Face Recognition Attendance System successfully automates attendance tracking with **high accuracy** and **efficiency**. It eliminates proxy attendance and reduces administrative effort.

6.2 Future Work

- **Improve Accuracy:** Use **deep learning models** (e.g., CNN).
 - **Mobile App Integration:** Allow attendance marking via smartphones.
 - **Cloud Database:** Store data on the cloud for remote access.
 - **Multi-Factor Authentication:** Combine face recognition with RFID for higher security.
-

7. Work Update

7.1 Work Completed

- **Student Registration Module** (GUI + MySQL).
- **Face Detection & Recognition** (Haar Cascade + LBPH).
- **Attendance Logging** (CSV export/import).
- **Training Module** (LBPH model training).

7.2 Work Remaining

- **Real-time alert for unrecognized faces.**
 - **Advanced reporting dashboard** (graphs/analytics).
 - **Integration with college ERP systems.**
-

8. References

1. OpenCV Documentation. <https://docs.opencv.org/>
2. LBPH Algorithm Paper: *Ahonen, T., Hadid, A., & Pietikäinen, M. (2004). "Face Recognition with Local Binary Patterns."*
3. MySQL Documentation. <https://dev.mysql.com/doc/>
4. Python Tkinter Guide. <https://docs.python.org/3/library/tkinter.html>

Test Metrics

Results:

Face Detection Report:

- Precision: 0.89
- Recall: 0.92
- F1-Score: 0.90

LBPH Recognition Accuracy: 87.5%

FPS: 12.3

Noise Robustness: 8% drop (Gaussian)

Database Query: 0.32 sec

1. Testing Framework

Key Metrics to Evaluate:

Category	Metrics	Tools/Methods
Accuracy	Face detection rate, Recognition rate	Confusion-Matrix, Precision/Recall
Speed	FPS (Frames Per Second)	time module (Python)

Category	Metrics	Tools/Methods
Robustness	Performance under noise/lighting	Synthetic noise injection
Database	Query response time	MySQL EXPLAIN ANALYZE
Usability	GUI responsiveness	User testing (manual)

2. Test Cases & Metrics

A. Face Detection (Haar Cascade)

Test 1: Detection Accuracy

- **Method:**

- Use a labeled dataset (e.g., 100 images with faces).
- Run Haar Cascade to count true/false positives/negatives.

- **Code:**

python

Copy

Download

import cv2

from sklearn.metrics import classification_report

face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

y_true = [] # Ground truth (1=face, 0=no face)

y_pred = [] # Model predictions

for img_path in dataset:

 img = cv2.imread(img_path)

 gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

 faces = face_cascade.detectMultiScale(gray, 1.1, 5)

 y_pred.append(1 if len(faces) > 0 else 0)

 y_true.append(label) # 1 if image has a face, else 0

print(classification_report(y_true, y_pred))

- **Metrics:**

- **Precision:** $TP / (TP + FP)$ (False alarms)
- **Recall:** $TP / (TP + FN)$ (Missed faces)
- **F1-Score:** Harmonic mean of precision/recall.

Test 2: Speed (FPS)

- **Code:**

python

Copy

Download

import time

```
cap = cv2.VideoCapture(0)
```

```
start_time = time.time()
```

```
frame_count = 0
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    faces = face_cascade.detectMultiScale(gray, 1.1, 5)
```

```
    frame_count += 1
```

```
    if frame_count % 100 == 0:
```

```
        fps = frame_count / (time.time() - start_time)
```

```
        print(f'FPS: {fps:.2f}')
```

- **Metric:**

- **FPS ≥ 10** for real-time use.

B. Face Recognition (LBPH)

Test 3: Recognition Accuracy

- **Method:**

- Train LBPH on 80% of data/ and test on 20%.

- **Code:**

python

Copy

Download

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(faces, labels, test_size=0.2)
```

```
# Train LBPH
```

```
model = cv2.face.LBPHFaceRecognizer_create()
```

```
model.train(X_train, np.array(y_train))
```

```
# Test
```

```

y_pred = []
for face in X_test:
    label, confidence = model.predict(face)
    y_pred.append(label)
print(f'Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%')

```

- **Metrics:**

- **Accuracy:** % of correctly recognized faces.
- **Confidence Scores:** Avg. confidence of correct predictions.

Test 4: Robustness to Noise

- **Method:**

- Add Gaussian/salt-and-pepper noise to test images.

- **Code:**

python

Copy

Download

```
import numpy as np
```

```

def add_noise(image, noise_type="gaussian"):
    if noise_type == "gaussian":
        row, col = image.shape
        mean = 0
        var = 0.1
        sigma = var ** 0.5
        gauss = np.random.normal(mean, sigma, (row, col))
        noisy = np.clip(image + gauss * 255, 0, 255).astype(np.uint8)
    elif noise_type == "s&p":
        # Salt-and-pepper logic
        pass
    return noisy

```

```

noisy_img = add_noise(gray_image)
label, confidence = model.predict(noisy_img)

```

- **Metric:**

- **Accuracy drop** under noise (e.g., 85% → 70%)